

JOBSHEET 15

Collections



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

15.1 Tujuan Praktikum

Tujuan Praktikum Setelah melakukan praktikum ini, mahasiswa mampu:

1. memahami manfaat Java Collection Framework;
2. mengimplementasikan class dalam Java Collection Framework sesuai kasus;

15.2 Persiapan

15.2.1 Langkah-langkah Percobaan

Class Customer:

```
package Jobsheet15;

public class Customer {
    public int id;
    public String name;

    public Customer() {
    }

    public Customer(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return " ID: " + this.id + " Nama: " + this.name;
    }
}
```

Class Book:

```
package Jobsheet15;

public class Book {
    public String isbn;
    public String title;

    public Book() {
    }

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String toString() {
        return "ISBN: " + this.isbn + " Title: " + this.title;
    }
}
```

```
}  
}
```

15.3 Praktikum - Implementasi ArrayList

15.3.1 Langkah – langkah percobaan

Class DemoArrayList:

```
package Jobsheet15;  
  
import java.util.ArrayList;  
  
public class DemoArrayList {  
    public static void main(String[] args) {  
        ArrayList<Customer> customers = new ArrayList<>();  
  
        Customer customer1 = new Customer(1, "Zaki");  
        Customer customer2 = new Customer(5, "Budi");  
  
        customers.add(customer1);  
        customers.add(customer2);  
  
        customers.add(new Customer(4, "Cica"));  
  
        customers.add(2, new Customer(100, "Rosa"));  
  
        System.out.println(customers.indexOf(customer2));  
  
        Customer customer = customers.get(1);  
        System.out.println(customer.name);  
        customer.name = "Budi Utomo";  
  
        ArrayList<Customer> newCustomers = new ArrayList<>();  
        newCustomers.add(new Customer(201, "Della"));  
        newCustomers.add(new Customer(202, "Victor"));
```

```

        newCustomers.add(new Customer(203, "Sarah"));

        customers.addAll(newCustomers);

        System.out.println(customers);

        for (Customer cust : customers) {
            System.out.println(cust.toString());
        }
    }
}

```

15.3.2 Verifikasi Hasil Percobaan

```

1
Budi
[ ID: 1 Nama: Zaki, ID: 5 Nama: Budi Utomo, ID: 100 Nama: Rosa, ID: 4 Nama: Cica, ID: 201 Nama: Della, ID: 202 Nama: Victor, ID: 203 Nama: Sarah]
ID: 1 Nama: Zaki
ID: 5 Nama: Budi Utomo
ID: 100 Nama: Rosa
ID: 4 Nama: Cica
ID: 201 Nama: Della
ID: 202 Nama: Victor
ID: 203 Nama: Sarah

```

15.3.3 Pertanyaan!

1. Cobalah tambahkan object customer baru ke dalam customers. Apakah object dapat ditambahkan meskipun melebihi kapasitas?

```

ArrayList<Customer> customers = new ArrayList<>(2);

Customer customer1 = new Customer(1, "Zakia");
Customer customer2 = new Customer(5, "Budi");

customers.add(customer1);
customers.add(customer2);

```

Jawaban:

Iya.

2. Compile dan run kode program, di mana object yang baru ditambahkan? Di awal, di tengah, atau di akhir collection?

Jawaban:

Di akhir collection.

3. Compile dan run kode program. Index pada ArrayList dimulai dari 0 atau 1?

Jawaban:

Index dimulai dari 0, dibuktikan dengan penambahan customer dengan id 100 yang berada pada index 2.

4. Cobalah hapus angka 2 saat instansiasi object customers. Apakah ArrayList dapat diinstansiasi tanpa harus menentukan size di awal?

Jawaban:

Bisa, karena ArrayList adalah array dinamis.

15.4 Praktikum - Implementasi Stack

15.4.1 Langkah-Langkah:

Class StackDemo:

```
package Jobsheet15;

import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Book book1 = new Book("1234", "Dasar Pemrograman");
        Book book2 = new Book("7145", "Hafalah Shalat Delisa");
        Book book3 = new Book("3562", "Muhammad Al-Fatih");

        Stack<Book> books = new Stack<>();
        books.push(book1);
        books.push(book2);
        books.push(book3);

        Book temp = books.peek();

        if (temp != null) {
            System.out.println(temp.toString());
        }

        Book temp2 = books.pop();

        if (temp2 != null) {
            System.out.println(temp.toString());
        }

        for (Book book : books) {
            System.out.println(book.toString());
        }

        System.out.println(books);

        System.out.println(books.search(book1));
    }
}
```

15.4.2 Verifikasi Percobaan

```
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 3562 Title: Muhammad Al-Fatih
ISBN: 1234 Title: Dasar Pemrograman
ISBN: 7145 Title: Hafalah Shalat Delisa
[ISBN: 1234 Title: Dasar Pemrograman, ISBN: 7145 Title: Hafalah
Shalat Delisa]
2
```

15.4.3 Pertanyaan

1. Mengapa perlu ada pengecekan (temp != null)?
Jawaban:
Karena diperlukan untuk mencegah ketika kondisi temp itu null, maka akan menyebabkan error NullPointerException.
2. Bagaimana cara melakukan pencarian elemen pada stack menggunakan method search()?
Jawaban:
Cara menggunakannya yaitu dengan menambahkan '.' Diikuti dengan method search(), lalu mengisi parameter dengan object yang ingin dicari.
Contoh jika ingin mencetak: System.out.println(books.search(book1));

15.5 Praktikum - Implementasi TreeSet

Class TreeSetDemo:

```
package Jobsheet15;

import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<String> fruits = new TreeSet<>();

        fruits.add("Mangga");
        fruits.add("Apel");
        fruits.add("Jeruk");
        fruits.add("Jambu");

        for (String temp : fruits) {
            System.out.println(temp);
        }

        System.out.println("First: " + fruits.first());
        System.out.println("Last: " + fruits.last());

        fruits.remove("Jeruk");
        System.out.println("Setelah remove " + fruits);
    }
}
```

```

        fruits.pollFirst();
        System.out.println("Setelah poll first " + fruits);
    }
}

```

15.5.1 Verifikasi Percobaan

```

Apel
Jambu
Jeruk
Mangga
First: Apel
Last: Mangga
Setelah remove [Apel, Jambu, Mangga]
Setelah poll first [Jambu, Mangga]

```

15.5.2 Pertanyaan

1. Compile dan run program. Mengapa urutan yang ditampilkan berbeda dengan urutan penambahan data ke dalam TreeSet fruits?

Jawaban:

Karena TreeSet secara otomatis mengurutkan elemen yang disimpan berdasarkan urutan ascending.

2. Apa yang dilakukan oleh method first(), last(), remove(), pollFirst(), dan pollLast()?

Jawaban:

first(): mengembalikan urutan pertama/element terkecil di dalam TreeSet.

last(): mengembalikan urutan terakhir/element terbesar di dalam TreeSet.

remove(): menghapus element tertentu yang ada pada parameter.

pollFirst(): menghapus urutan pertama di dalam TreeSet.

15.6 Praktikum – Sorting

Class SortDemo:

```
package Jobsheet15;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
public class SortDemo {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> daftarSiswa = new ArrayList<>();
```

```
        daftarSiswa.add("Zainab");
```

```
        daftarSiswa.add("Andi");
```

```
        daftarSiswa.add("Rara");
```

```
        Collections.sort(daftarSiswa);
```

```

        System.out.println(daftarSiswa);

        ArrayList<Customer> customers = new ArrayList<>();
        customers.add(new Customer(201, "Della"));
        customers.add(new Customer(202, "Victor"));
        customers.add(new Customer(203, "Sarah"));

        customers.sort((c1,c2) -> c1.name.compareTo(c2.name));
        System.out.println(customers);
    }
}

```

15.6.1 Verifikasi Percobaan

```

[Andi, Rara, Zainab]
[ ID: 201 Nama: Della, ID: 203 Nama: Sarah, ID: 202 Nama: Victor]

```